

A Comparative Study of Large Language Models for Model-Driven Reverse Engineering

Hamza Abdelmalek

GL-ISI Team

Faculty of Sciences and Techniques of Errachidia, UMI

Errachidia, Morocco

h.abdelmalek@edu.umi.ac.ma

Zakaria Babaalla

GL-ISI Team

Faculty of Sciences and Techniques of Errachidia, UMI

Errachidia, Morocco

za.babaalla@edu.umi.ac.ma

Abdeslam Jakimi

GL-ISI Team

Faculty of Sciences and Techniques of Errachidia, UMI

Errachidia, Morocco

ajakimi@yahoo.fr

Rachid Saadane

SIRC-LAGES, Electrical Engineering Department

Hassania School of Public Works

Casablanca, Morocco

saadane@ehp.ac.ma

Abstract—The extraction of abstract models from the source code is one of the main focuses of model-driven reverse engineering and is particularly important for legacy systems, which do not have updated documentation. Recovering abstract models from legacy systems is complex, given that the technical constructs combined with domain-level constructs are implemented through multiple layers of technology and system-specific rules. Current static analysis and heuristic-based approaches can struggle to separate these concerns. The recent emergence of large language models (LLMs) is promising with regard to modeling software, but behaviors and expected outcomes related to LLMs when performing reverse engineering tasks are challenging. This paper compares the performance of various LLMs on a reverse engineering task related to the recovery of abstract models from the source code. This comparison aims to identify the domain constructs represented through an intermediate representation of the source code. Three LLMs, namely ChatGPT, Claude, and Meta AI, were assessed on the same input representation, with each LLM performing the task of recovering abstract models through the use of a common prompt. Evaluation of the three LLM outputs was performed on a common open-source C# project, which serves as the target for the LLM outputs to be evaluated against an established reference Platform-Independent Model. Evaluation of model outputs was performed according to precision, recall, and F1 score, measuring individual categories of model elements.

Index Terms—large language models, model-driven architecture, model-driven reverse engineering, model recovery, platform-independent model

I. INTRODUCTION

Model-Driven Engineering (MDE) places models as first-class artifacts that capture system abstractions and domain knowledge. It supports systematic analysis and automation throughout the software development lifecycle. Model-Driven Architecture (MDA) is an implementation of MDE that promotes the separation of domain logic from platform and implementation concerns through the use of abstract models, particularly the Platform-Independent Model (PIM) [1].

In practice, many software systems were developed without explicit models or with models that are no longer maintained, especially legacy systems [2]. As a result, understanding, evolving, or modernizing such systems often requires recovering high-level domain abstractions directly from source code. This recovery task is difficult because domain concepts are coupled with technical infrastructure, framework code, and platform-specific details [3].

Recovering a PIM from existing code is more challenging than forward modeling because the original domain intent is not explicitly represented. Reverse engineering must infer abstractions that were never formally modeled and must distinguish between domain-relevant concepts and purely technical constructs. Existing model-driven reverse engineering (MDRE) approaches rely heavily on static analysis artifacts, such as abstract syntax trees (ASTs), and on rule-based or heuristic transformations [4]. Although these methods offer structure and determinism, they often struggle with scalability and with capturing implicit domain semantics.

Recent advances in large language models (LLMs) have raised interest in their potential use for software modeling tasks [5]. LLMs can interpret code and textual descriptions and can generate structured outputs such as Unified Modeling Language (UML) models [6]. By directly identifying domain constructs from code representations, LLMs have the potential to accelerate model recovery. However, hallucination, irregular structure, and output quality variability are new issues introduced by LLM-based extraction.

This paper investigates the use of LLMs for PIM recovery from source code within an MDRE context, to compare how different LLMs perform under identical inputs, instructions, and evaluation criteria rather than propose a new toolchain or integration framework. The study focuses on recovering PIM elements from a system's source code using LLMs. The proposed approach parses the source code into an AST and then transforms it into a compact intermediate representation

979-8-3195-1831-6/26/\$31.00 ©2026 IEEE

(IR) that retains only modeling-relevant information. This representation is provided to each LLM using a single, identical prompt that requests all PIM elements in one step. The use of a single prompt and a single execution per model prioritizes transparency and reproducibility and reflects common usage patterns of chat-based LLM interfaces.

A single open-source case study implemented in C# is used for the evaluation. In this context, the ground truth consists of a manually constructed reference PIM. Based on this reference, precision, recall, and F1-score are computed for each category of model element to compare the outputs generated by each model. The findings show that the assessed LLMs exhibit varying trade-offs between over-inclusion and omission. While attributes and operations are typically recovered with high accuracy, classifier and relationship extraction continue to be more prone to errors. These observations illustrate the benefits and drawbacks of current LLMs when viewed in the context of model recovery.

The remainder of the paper is organized as follows. Section 2 reviews related work on model recovery and AI-based MDRE. Section 3 describes the methodology. The results and the discussion are presented in Section 4. The paper is concluded, and future research directions are outlined in Section 5.

II. RELATED WORK

A. Model Recovery

The PIM in MDA depicts the system's domain concepts and their interactions, while abstracting away implementation and platform details. This separation is beneficial because it facilitates cross-platform reuse and maintains domain logic stability as technologies evolve. Domain concepts are mixed with infrastructure concerns, persistence code, and framework conventions [7]. As a result, recovering a PIM from code is generally harder than producing one in forward engineering, because the recovery process must filter technical noise and reconstruct abstractions that were never modeled explicitly [8].

According to Marcén et al., tool support and evaluation techniques differ greatly between studies, and many MDRE pipelines rely on transformations and heuristics to bridge the gap between code and model abstractions [9]. In practice, recovery techniques frequently begin with static code analysis before generating more abstract representations through model transformations. Accordingly, the C2AADL-Reverse tool uses a model-driven process to extract architectural models for safety-critical systems, showing how intermediate artifacts can help ensure consistent extraction of higher-level views [10]. According to Assunção et al., Many modernization pipelines combine reverse engineering with refactoring, migration, and re-architecting activities, typically requiring the identification of domain boundaries and core abstractions before transformation [11].

B. AI for MDE

To recover models from software artifacts, AI-based methods have been investigated. In our recent work, we discussed

integrating MDE with clustering to support the comprehension and restructuring of legacy systems [12]. In addition, LLMs have recently been proposed for tasks more closely related to software modeling. In this regard, a review of recent work on LLMs for software engineering highlights that LLM-based methods encompass a range of tasks, such as software maintenance and code generation, with recurring problems pertaining to evaluation methodology and reproducibility [13]. In this sense, Babaalla et al. proposed an LLM-driven MDA approach to transform textual specifications into source code by combining transformer-based semantic extraction with a domain-specific language as an IR [14]. In contrast, another study found that LLM-generated artifacts often require verification, and that quality is affected by how users prompt and integrate LLM outputs [15].

Several recent studies investigate LLMs in UML modeling and provide information about possible failure scenarios for model extraction. Bari et al. state that performance varies according to the type of modeling decision required, and maintaining completeness is challenging when the specification is unclear or incomplete [16]. Related comparative work evaluates different LLMs for generating UML class diagrams from textual requirements and discusses common error patterns among models as well as differences in diagram quality [17]. Recently, MDE pipelines have increasingly incorporated LLMs. For example, Siala and Lano propose an approach where they extract Object Constraint Language specifications from Java and Python source code using LLMs [18].

III. METHODOLOGY

A. Study Design

The study's goal is to compare how different LLMs perform on the same MDRE task under the same conditions, as illustrated in Figure 1. For this reason, all experiments use direct chat-based interfaces provided by each LLM. To further reduce confounding factors, the scope of the study is restricted to the recovery of the PIM in MDA. In addition, the experiment is conducted on a single case study. This design decision prevents variability caused by project size, architectures, or coding styles and allows for a clearer analysis of the differences between LLMs. The selected system contains a sufficient number of domain concepts to support meaningful evaluation while remaining manageable for manual inspection and validation. In addition, the LLMs were evaluated using the same IR extracted from the AST and the same prompt.

The ground-truth PIM was manually created by examining the source code of the case study. First, classes were initially discovered by examining namespaces, class names, and their roles in the core functions of the system. The classes that were clearly technical, auxiliary, or involved low-level implementation details were excluded. Second, attributes representing domain state were discovered for each selected classifier using explicitly declared fields and properties. Third, operations that represent business behavior, constructors, accessors, and purely technical methods were removed from the method declaration analysis. Finally, relationships were created between

the identified classifiers, including associations, compositions, aggregations, and inheritance relationships. The resulting reference PIM serves as the baseline for evaluating the outputs produced by the LLMs. Precision, recall, and F1-score are computed by comparing the extracted elements against this manually constructed PIM.

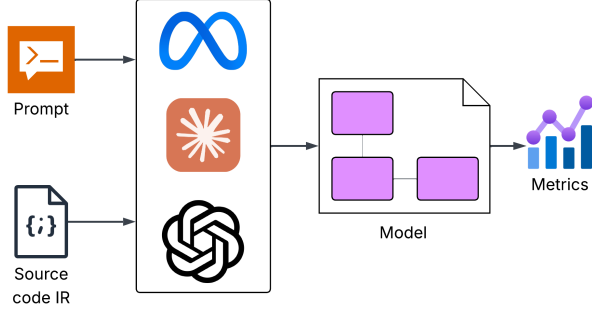


Fig. 1. Study Design for LLM-Based Model Recovery

B. Case Study

The case study is an open-source blockchain implementation written in C# called BlockchainSharp [19]. The project implements cryptographic hashes, blocks, transactions, and basic peer-to-peer communication methods. The codebase contains 46 classifiers and approximately 2,800 lines of code. As a result, BlockchainSharp is suitable for controlled experiments aimed at conceptual model recovery due to its relatively simple object-oriented design. Two primary criteria were used in the selection of this case study. Firstly, it includes immediately recognizable domain concepts like blocks, transactions, and addresses that are linked to PIM elements. Secondly, manual validation and inspection are feasible without complexity due to the project's moderate size.

C. Experimental Setup

The first step in the recovery process is to parse the source code into an AST. The AST provides a complete and accurate representation of the program's syntax. However, ASTs contain a lot of low-level details that are irrelevant. To overcome these limitations, the AST is transformed into a compact IR, which is designed to preserve only information relevant to model recovery.

Three LLMs from OpenAI's GPT¹, Claude², and Meta AI's Llama³ are included in the study. These models were chosen to represent various LLM families, enable a fair comparison, and preserve a constrained and intelligible experimental scope. To prevent redundancy, models belonging to the same family or slight variations of the same architecture were not included. The extraction process was performed once for every model using the same IR of the source code and a single prompt. The

study avoids dependencies between intermediate outputs by employing a single prompt, which guarantees that every model receives the same input and instructions without variation across stages. This approach also reflects how LLMs are commonly used in practice through interactive chat interfaces, where users often issue a single, comprehensive request rather than a sequence of coordinated prompts.

D. Evaluation Metrics

Every LLM's output is directly compared to the reference model. Precision, recall, and F1-score are the standard metrics used for each PIM element category. Various facets of extraction quality are measured by these metrics. Precision quantifies the percentage of correctly extracted elements as defined in equation "(1)", recall quantifies the percentage of successfully extracted reference elements as defined in equation "(2)", and the F1-score offers a mean summary of both metrics as defined in equation "(3)" [20]. where TP, FP, and FN denote true positives, false positives, and false negatives, respectively.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

Classifier evaluation is performed by comparing the set of extracted classifier names against the set defined in the reference PIM. Both lists are normalized by converting names to lowercase and trimming whitespace, and they are treated as unordered sets. Attribute and operation evaluation is performed at the classifier level. Only classifiers that appear in both the reference and generated models are considered. For each classifier, attributes and operations are compared as sets after normalization. Metrics are first computed per classifier and then aggregated using micro-averaging. Classifiers for which both the reference and generated models contain no attributes or operations are excluded. Relationship evaluation follows a similar comparison but applies additional matching rules depending on the relationship type. For inheritance, aggregation, and composition relationships, the direction is preserved, and relationships must match exactly. For association relationships, direction is ignored, and two relationships are considered equivalent if they connect the same pair of classifiers with matching multiplicities, regardless of order.

IV. RESULTS AND DISCUSSION

A. Results

Table I summarizes each model's precision, recall, and F1-score for each PIM element category. With a perfect precision of 1.00 and a lower recall of 0.58, ChatGPT achieved an F1-score of 0.73 for classifier extraction. Claude showed a different profile with an F1-score of 0.73, a higher recall of 0.84, and a lower precision of 0.64. Meta AI exhibited the lowest classifier precision (0.48) but the same recall (0.84) as

¹<https://openai.com/>

²<https://claude.ai/>

³<https://www.meta.ai/>

TABLE I
PRECISION, RECALL, AND F1-SCORE RESULTS FOR PIM RECOVERY USING LLMs

Project	Model	Model element	Precision	Recall	F1-score
BlockchainSharp	ChatGPT	Classifiers	1.00	0.58	0.73
		Attributes	1.00	0.75	0.86
		Operations	1.00	0.67	0.80
		Relationships	0.83	0.45	0.58
	Claude	Classifiers	0.64	0.84	0.73
		Attributes	1.00	0.75	0.86
		Operations	1.00	1.00	1.00
		Relationships	0.58	0.60	0.59
	Meta AI	Classifiers	0.48	0.84	0.61
		Attributes	1.00	0.87	0.93
		Operations	1.00	1.00	1.00
		Relationships	0.40	0.77	0.53

Claude, with an F1-score of 0.61. These results imply that the main distinctions between classifier models are driven by the trade-off between conservative selection and over-inclusion.

Attribute extraction produced consistently high precision across all three models. Differences appeared mainly in recall. ChatGPT and Claude achieved the same recall of 0.75 and an F1-score of 0.86, while Meta AI achieved a higher recall of 0.87 and the best attribute F1-score of 0.93. At the classifier level, missed attributes were concentrated in a small number of classes.

Regarding operations, ChatGPT again showed perfect precision but a lower recall of 0.67, resulting in an F1-score of 0.80. Claude and Meta AI achieved perfect recall and precision, resulting in an F1-score of 1.00. The disparity can be explained by omissions from some classifiers in ChatGPT’s output. For example, ChatGPT missed calculatehash in the transaction and failed to extract operations for blockhash, resulting in a total of four false negatives. This omission is consistent with ChatGPT’s conservative filtering strategy, which appears to have treated hash computation as a technical infrastructure concern rather than a domain-level operation. However, under the chosen comparison rules, Claude and Meta AI produced zero false negatives by matching the reference operations for every classifier that was evaluated.

Relationship extraction yielded the lowest F1-scores of all the models, as clearly shown in Figure 2, because relationships are more structurally sensitive than isolated elements like attributes. ChatGPT produced a precision of 0.83 and a recall of 0.45. Claude achieved a similar balance with an F1-score of 0.59. Meta AI produced the highest recall (0.77) but the lowest precision (0.40) with an F1-score of 0.53. These results suggest that the main distinction between the models was whether they preferred a more expansive relationship set with many false links or a conservative relationship set.

To illustrate the types of errors observed, ChatGPT produced two false positive associations: (Block, BlockHeader) and (BlockHeader, Hash). In the reference PIM, these are modeled as compositions rather than associations, indicating that ChatGPT failed to distinguish between relationship types when the IR did not explicitly encode the distinction. Conversely, ChatGPT missed all relationships involving classifiers it had not extracted, such as BlockMessage, StatusMessage, and

TransactionMessage, which contributed to 12 false negatives. Claude and Meta AI recovered more reference relationships by including these classifiers, but this broader inclusion also led to false positives

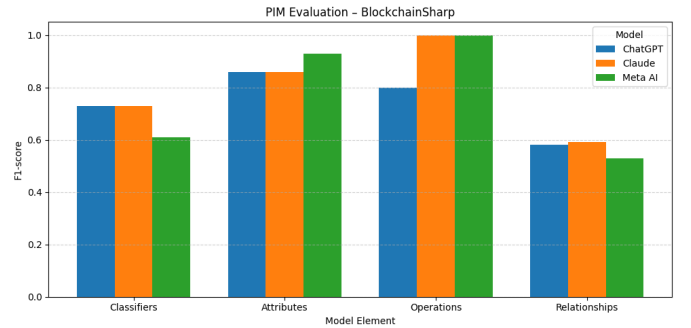


Fig. 2. F1-Score Results for PIM Element Recovery Using LLMs

B. Discussion

1) *LLMs’ Strengths and Limitations:* Across all three LLMs, attribute and operation extraction achieved high precision. In particular, no model introduced false positive attributes or operations, and both Claude and Meta AI recovered the full set of reference operations. This suggests that when the IR provides explicit signatures and declared members, models can extract these elements reliably under a strict prompt. The results also show that a structured IR helps the models focus on modeling-relevant information [21]. Even when models differed in how many classifiers they included, they still produced coherent internal descriptions of the classifiers they selected. For example, once the transaction class was identified, the extracted attributes and operations were largely aligned with the reference model. This pattern supports the use of a simplified IR as a practical input format for early-stage conceptual modeling.

The main limitations appear in classifier selection and, more strongly, relationship recovery. The three LLMs exhibited different error profiles, which can be interpreted as different strategies for handling the prompt’s conservativeness constraint. ChatGPT produced no false positive classifiers, but it missed eight reference classifiers. This indicates a conservative

filtering strategy in which the model prefers to omit elements that may appear technical. The same conservative behavior affected recall for operations and relationships. In practice, this behavior reduces the need for cleaning incorrect elements, but it increases the amount of manual completion required.

Recall was enhanced by Claude's recovery of the majority of reference classifiers, but it also added a number of extra components not found in the reference PIM. Relationships were also impacted by this behavior because more associations are made possible by each additional classifier. The highest capacity to over-include was demonstrated by Meta AI. Due to a greater number of false positives, it had significantly lower precision, even though it had the same classifier recall as Claude. The same pattern for relationships, with Meta AI generating a large number of new associations. Although these outputs add a significant amount of noise, they improve recall for relationships. When coverage is the main objective, this behavior can be helpful in practice; however, technical artifacts must be carefully removed by hand.

2) *Implications for MDE Practice:* The results suggest that the choice of LLM depends on the intended use of the extracted model. If the goal is to obtain a conservative model that can be used as a starting point with minimal incorrect elements, ChatGPT's behavior is closer to that objective, but it may require manual completion to address omissions. If the goal is broader coverage of candidate elements, Claude and Meta AI provide higher recall for classifiers and relationships, but the resulting models require more filtering to remove technical constructs that are not part of the intended model scope.

The results also indicate that LLMs can produce usable outputs for early modeling tasks [5], especially for attributes and operations. However, relationships remain a major source of error. For practical use, this suggests that human validation should focus first on classifier scope, since classifier over-inclusion tends to propagate to relationship over-generation. A lightweight review step that removes infrastructure classes before validating relationships is likely to reduce noise in the final model.

V. CONCLUSION AND FUTURE WORK

This study examined how well different LLMs performed in recovering the PIM from source code. The study compared ChatGPT, Claude, and Meta AI using a single case study. The evaluation focused on measuring precision, recall, and F1-score for classifiers, attributes, operations, and relationships model elements.

The findings show that the three LLMs have different inclusion and omission trade-offs. Despite missing a significant percentage of the reference classifiers and relationships, ChatGPT generated the cleanest classifier set, attaining perfect precision for classifiers, attributes, and operations. Classifiers and relationships saw an increase in recall but a decrease in precision as Claude introduced new platform concepts. Although Meta AI produced the most false positives for classifiers and relationships, it also had the highest recall

for attributes and relationships. Relationships were generally more difficult to recover than operations and attributes in all LLMs. Overall, the findings demonstrate that although LLMs can aid in the recovery of early-stage PIM from a structured representation, their outputs cannot be used interchangeably. Depending on the extraction behavior, different types and amounts of manual validation are required. Although a permissive model output improves coverage, non-domain artifacts must be eliminated. A conservative model output, on the other hand, reduces filtering effort but might need to be completed by hand.

Several avenues for further research are suggested by these findings. First, since the MDA's Platform Model (PM) involves more platform-specific constructs and may amplify differences in how models interpret technical elements, the evaluation should be extended to include the PM recovery. Second, a different kind of artifact, namely execution traces, must be analyzed to study the extraction of behavioral models, such as use cases and sequence diagrams [22]. Third, the relationship findings point to the necessity of better structural link management. Tighter relationship constraints, IR-based post-processing rules, or a condensed version of staged extraction that separates relationships from classifier selection while maintaining a straightforward overall strategy can all be investigated. Future work will extend the evaluation to multiple case studies of varying size, domain, and programming language to assess the generalizability of the observed patterns. Additionally, investigating the effect of prompt variations, including multi-step and iterative prompting strategies, on extraction quality is a relevant direction. Furthermore, a comparison with traditional rule-based and static analysis approaches for model recovery would help position LLM-based extraction relative to established MDRE techniques and is planned as a complementary study

REFERENCES

- [1] J. Miller and J. Mukerji, *MDA Guide Version 1.0.1*. Object Management Group, June 2003.
- [2] R. Khadka, B. V. Batlajery, A. M. Saeidi, S. Jansen, and J. Hage, "How do professionals perceive legacy systems and software modernization?," in *Proc. 36th Int. Conf. Software Engineering*, 2014.
- [3] H. Abdelmalek, G. Chénard, I. Khriiss, and A. Jakimi, "A bimodal approach for the discovery of a view of the implementation platform of legacy object-oriented systems under modernization process," in *Proc. CATA*, 2020, pp. 98–111, doi:10.29007/rbp7.
- [4] H. A. Siala, K. Lano, and H. Alfraihi, "Model-driven approaches for reverse engineering—A systematic literature review," *IEEE Access*, vol. 12, pp. 62558–62580, 2024.
- [5] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubei, "On the use of large language models in model-driven engineering," *Software and Systems Modeling*, pp. 1–26, 2025, doi:10.1007/s10270-025-01263-8.
- [6] Z. Babaalla, H. Abdelmalek, A. Jakimi, and M. Oualla, "Extraction of UML class diagrams using deep learning: Comparative study and critical analysis," *Procedia Computer Science*, vol. 236, pp. 452–459, 2024.
- [7] H. Abdelmalek, I. Khriiss, and A. Jakimi, "Towards an effective approach for composition of model transformations," *Frontiers in Computer Science*, vol. 6, p. 1357845, 2024, doi:10.3389/fcomp.2024.1357845.
- [8] P. Andre, "Case studies in model-driven reverse engineering," in *Proc. 7th Int. Conf. Model-Driven Engineering and Software Development (MODELSWARD)*, 2019, pp. 20–22.

- [9] A. C. Marcén, A. Iglesias, R. Lapeña, F. Pérez, and C. Cetina, "A systematic literature review of model-driven engineering using machine learning," *IEEE Transactions on Software Engineering*, 2024.
- [10] Z. Yang, Z. Qiu, Y. Zhou, Z. Huang, J.-P. Bodeveix, and M. Filali, "C2AADL_Reverse: A model-driven reverse engineering approach to development and verification of safety-critical software," *Journal of Systems Architecture*, vol. 118, 2021, doi:10.1016/j.sysarc.2021.102202.
- [11] W. K. Assunção, L. Marchezan, L. Arkoh, A. Egyed, and R. Ramler, "Contemporary software modernization: Strategies, driving forces, and research opportunities," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–35, 2025.
- [12] H. Abdelmalek, Z. Babaalla, C. Ouaddi, L. Benaddi, and A. Jakimi, "Model-driven engineering and machine learning for legacy system modernization," *Engineering, Technology & Applied Science Research*, vol. 16, no. 1, pp. 32285–32291, 2026.
- [13] X. Hou *et al.*, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [14] Z. Babaalla, A. Jakimi, and M. Oualla, "LLM-driven MDA pipeline for generating UML class diagrams and code," *IEEE Access*, 2025.
- [15] S. Rasnayaka, G. Wang, R. Shariffdeen, and G. N. Iyer, "An empirical study on usage and perceptions of LLMs in a software engineering project," in *Proc. 1st Int. Workshop on Large Language Models for Code*, 2024, pp. 111–118, doi:10.1145/3643795.3648379.
- [16] D. De Bari, G. Garaccione, R. Coppola, M. Torchiano, and L. Ardito, "Evaluating large language models in exercises of UML class diagram modeling," in *Proc. 18th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement*, 2024, pp. 393–399.
- [17] G. Garaccione, D. M. Calabrese, R. Coppola, and L. Ardito, "A comparison of different large language models for the generation of UML class diagrams," in *Proc. 28th ACM/IEEE Int. Conf. Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2025, pp. 541–545.
- [18] H. Siala and K. Lano, "Using LLMs to extract OCL specifications from Java and Python programs: An empirical study," in *OCL '25, STAF*, 2025.
- [19] Ajlopez, "BlockchainSharp," GitHub repository. Available: <https://github.com/ajlopez/BlockchainSharp>. Accessed 2026.
- [20] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," *arXiv preprint arXiv:2010.16061*, 2020.
- [21] M. DeLorenzo *et al.*, "Abstraction-of-Thought: Intermediate representations for LLM reasoning in hardware design," *arXiv preprint arXiv:2505.15873*, 2025.
- [22] H. Abdelmalek, I. Khriss, and A. Jakimi, "Towards a hybrid approach to reverse engineer behavioral UML diagrams from source code," *Journal of Theoretical and Applied Information Technology*, vol. 101, no. 22, 2023.